

CMPT 383 Quiz 4 (Haskell)

Summer 2026

Duration	30 minutes for the entire exam.
Coding Questions	For coding questions, use pseudocode in the style of the language the question is about.
Aids allowed	Pencil or pen, and eraser. No notes, no papers, no books, no computers, no calculators, no cheat sheets, etc.
During the exam	Raise your hand if you would like to speak with a proctor. To be fair to all students, we don't answer questions about the exam. Choose the answer based on your best understanding of the question. Re-reading the question may help.

1. For each expression, write the **most general type signature**. You **don't** need to write `::`, and don't worry about the Foldable type class (use list instead).

Expression	Most general type signature
head	<code>[a] -> a</code>
drop	<code>Int -> [a] -> [a]</code>
(+)	<code>Num a => a -> a -> a</code>
(++)	<code>[a] -> [a] -> [a]</code>
(.)	<code>(b -> c) -> (a -> b) -> a -> c</code>
filter	<code>(a -> Bool) -> [a] -> [a]</code>
map	<code>(a -> b) -> [a] -> [b]</code>
map head	<code>[[b]] -> [b] -- ok if a instead of b</code>
foldr (for lists)	<code>(a -> b -> b) -> b -> [a] -> b</code>
(==)	<code>Eq a => a -> a -> Bool</code>
flip	<code>(a -> b -> c) -> b -> a -> c</code>

2. What does it mean that Haskell uses **lazy evaluation**? Give a **small code example** of a significant thing Haskell can do because of lazy evaluation that languages without lazy evaluation cannot do.

Lazy evaluation means Haskell evaluates expressions as needed, e.g. when the function call `f a b c` is made, `a`, `b`, and `c` are passed to `f` **without** first being evaluated.

This lets Haskell do things like:

- Work with infinite lists, e.g. `take 5 [1..]` returns `[1,2,3,4,5]`.
- Write a macro-like function like `nand` or `myIf`, e.g.

```
myIf :: Bool -> a -> a -> a
```

```
myIf True  x y = x
```

```
myIf False x y = y
```

3. Write a function called `count x L` that returns the number of times `x` appears in list `L`. It should work with **as many different types as possible**. Here are a few examples:

`count 'a' "charlie"` returns 1

`count 4 [1,4,4,1,5,4]` returns 3

`count True [False,False,False,True,True]` returns 2

`count ["up","on"] [ ["up","bird"], ["on","up"], ["up","on"] ]` returns 1

Include the most general type signature in your answer. Also, to get full marks, your solution should **not** have an explicit recursive call to itself.

```
count :: Eq a => a -> [a] -> Int
count x = length . filter (== x)
```

Or:

```
count x = foldr (\y acc -> if x == y then acc + 1 else acc) 0
```

Here are some versions with explicit recursive calls (and so don't get full marks):

```
count _ [] = 0
count x (y:ys) = (if x == y then 1 else 0) + count x ys
-- count x (y:ys) = if x == y
--                  then 1 + count x ys
--                  else count x ys
```